Aprendizaje automático avanzado
Autograd

Juan Miguel Santos

Laboratorio de investigación y desarrollo experimental en computación (LIDEC)
Universidad Nacional de Hurlingham

(22 de octubre de 2026)

Autograd

Introducción

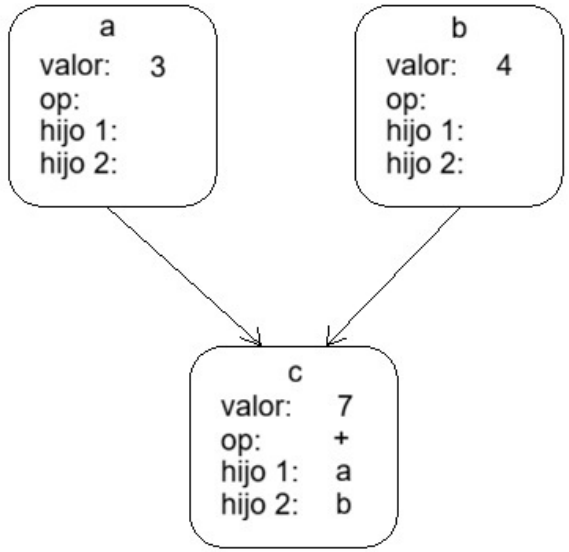
- Consideremos las siguientes ecuaciones:

$$a = 3$$
$$b = 4$$
$$c = a + b$$

- Lo que vamos a hacer es representar cada variable como un nodo de un grafo y vamos a unir los nodos de acuerdo a las dependencias que se establecen en las ecuaciones.

Autograd

Introducción



Grafo de las ecuaciones

$a = 3$

$b = 4$

$c = a + b$

Autograd

Introducción

- Lo que hicimos es construir una estructura (grafo) que nos permite guardar las relaciones entre las diferentes variables en las ecuaciones.
- En este ejemplo, a y b son hijos de c.

Autograd

Introducción

- Fijarse que en el nodo que corresponde a c se especifica la operación con que están relacionados sus hijos.
- a y b no tienen hijos ya que son asignaciones de un valor a una variable y por ese mismo motivo no hay una operación asignada.

Autograd

Introducción

- ¿Cómo podemos construir este grafo?
- Veamos primero cómo podemos representar un nodo en Python.

Autograd

Introducción

- Para esto definiremos una clase llamada Var:

```
class Var:
    def __init__(self, valor: int, nombre: str, _hijos = (), _op: str = ""):
        self.valor = valor
        self.nombre = nombre
        self._hijos = set(_hijos)
        self._op = _op
        def __repr__(self) → str:
            return f"({self.nombre}, = {self.valor})"
```

- La función __repr__() es usada por la función print() cuando se le solicita imprimir un elemento de la clase.

Autograd

Introducción

- Así, si quisiéramos definir un objeto de la clase Var, por ejemplo, a $= 3$ escribimos

  a = Var(3, "a")

- Si después quisiéramos imprimir al objeto "a" para conocer su valor, escribimos

  print(a)

  con resultado

  a = 3

Autograd

Introducción

- Si tenemos las siguientes variables

  a = Var(3, "a")
  b = Var(4, "b")

  y luego queremos expresar c como la suma entre a y b

  c = a + b

  obtendríamos un error
  > (Error)

  TypeError: unsupported operand type(s) for +: 'Var' and 'Var'

- Esto se debe a que no está definida la suma ("+") entre dos objetos de la clase Var.

Autograd

Introducción

- De modo que debemos definir la suma dentro de la clase Var:

```
def __add__(self, otro_operando):
    resultado = Var(valor = self.valor + otro_operando.valor, nombre=,
                    _hijos = (self, otro_operando), _op = "+")

return resultado
```

Autograd

Introducción

- Si ahora probamos nuevamente:

```
a = Var(3, "a")
b = Var(4, "b")
c = a + b
c.nombre = "c"
print(c)
```

el resultado es

```
c = 7
```

Autograd

Introducción

- Sin embargo hay dos detalles más para resolver. El primero es cuando deseamos escribir:

  a = Var(5, "a")
  b = a + 3
  b.nombre = "b"

  que resulta en un error:

  > Error:

  AttributeError: 'int' object has no attribute 'valor'

Autograd

Introducción

- Así que ajustamos la definición de __add__().
- __add__() es la forma de operar para Python cuando en un programa se usa el signo "+" para expresar adición:

```
def __add__(self, otro_operando):
    # si otro_operando no es un objeto de la clase Var
    # Convertir otro_operando a un objeto de la clase Var
    if not isinstance(otro_operando, Var):
        otro_operando = Var(valor = otro_operando,
                            nombre = str(otro_operando))
    resultado = Var(valor = self.valor + otro_operando.valor, nombre = "",
                    _hijos = (self, otro_operando), _op = "+")
    return resultado
```

Autograd

Introducción

- Ahora, si procedemos nuevamente:

```
a = Var(5, "a")
b = a + 3
b.nombre = "b"
print(b)
```

obtenemos:

```
b = 8
```

Autograd

Introducción

- El otro problema que tendremos es cuando deseamos escribir:

```
a = Var(5, "a")
b = 3 + a
b.nombre = "b"
print(b)
```

- Nuevamente nos devolvería error ya que la constante "3" es el operando de la izquierda (que se pasa como self) y no es de la clase Var.

Autograd

Introducción

- Para esto procedemos recurriendo a re escribir una definición por defecto de __add__() que es __radd__().
- Cuando __add__() falla, Python llama a __radd__() intercambiando los argumentos para ver si se soluciona el problema:

```
def __radd__(self, otro_operando):
    return self + otro_operando
```

Autograd

Resumiendo

```
class Var:
    def __init__(self, valor: int, nombre: str, _hijos = (), _op: str = ""):
        self.valor = valor
        self.nombre = nombre
        self._hijos = set(_hijos)
        self._op = _op
    def __repr__(self) → str:
        return f"(self.nombre, = self.valor)"
    def __add__(self, otro_operando):
        # si otro_operando no es un objeto de la clase Var
        # Convertir otro_operando a un objeto de la clase Var
        if not isinstance(otro_operando, Var):
            otro_operando = Var(valor = otro_operando,
                                nombre = str(otro_operando))
        resultado = Var(valor = self.valor + otro_operando.valor, nombre = "",
                        _hijos = (self, otro_operando), _op = "+")
        return resultado
    def __radd__(self, otro_operando):
        return self + otro_operando
```

Autograd

Introducción

- Una definición más completa de la clase Var incluye nuevas operaciones tales como multiplicación, potencia, división y el operador unario "-":

```
# Multiplicación
def __mul__(self, otro_operando):
    if not isinstance(otro_operando, Value):
        otro_operando = Var(valor = otro_operando,
                            nombre = str(otro_operando))
    resultado = Var(valor = self.valor * otro_operando.valor, nombre = "",
                    _hijos = (self, otro_operando), _op = "*")
    return resultado
def __rmul__(self, otro_operando):
    return self * otro_operando
```

Autograd

Introducción

- Una definición más completa de la clase Var incluye nuevas operaciones tales como multiplicación, potencia, división y el operador unario "-":

```
# Menos unario
def __neg__(self):
    return self * -1
# Substracción (Caso especial de adición)
def __sub__(self, otro_operando):
    return self + (-otro_operando)
def __rsub__(self, otro_operando):
    return otro_operando + (-self)
```

Autograd

Introducción

- Una definición más completa de la clase Var incluye nuevas operaciones tales como multiplicación, potencia, división y el operador unario "-":

```
# Potencia
def __pow__(self, otro_operando):
    resultado = Var(valor = self.valor**otro_operando, nombre = ,
                    _hijos = (self,), _op = f"**{otro_operando}")
    return resultado
# División (Caso especial de multiplicación con un operando expresado a la -1)
def __truediv__(self, otro_operando):
    return self * otro_operando ** -1
def __rtruediv__(self, otro_operando):
    return otro_operando * self**-1
```

Autograd

Introducción

- Incluso, es necesario definir algunas funciones que serán importantes para redes neuronales artificiales:

```
# Funciones de activación
# Exponencial
def exp(self):
    resultado = Var(valor = math.exp(self.valor),
                    nombre = f"exp({self.valor})", _hijos = (self,), _op =
                    "exp")
    return resultado
# Función Tanh
def tanh(self):
    este_valor = (math.exp(2 * self.valor) - 1) / (math.exp(2 * self.valor) + 1)
    resultado = Var(este_valor, nombre = f"tanh({self.nombre })", _op =
                    "tanh", _hijos=(self, ))
    return resultado
```

Autograd

Introducción

- Incluso, es necesario definir algunas funciones que serán importantes para redes neuronales artificiales:

```
# Funciones de activación
# ReLU (Rectified Linear Unit function)
def relu(self):
    este_valor = 0 if self.valor ≤ 0 else self.valor
    resultado = Var(este_valor, nombre = f"relu({self.nombre })", _op =
                    "relu", _hijos = (self, ))
    return resultado
```

Autograd

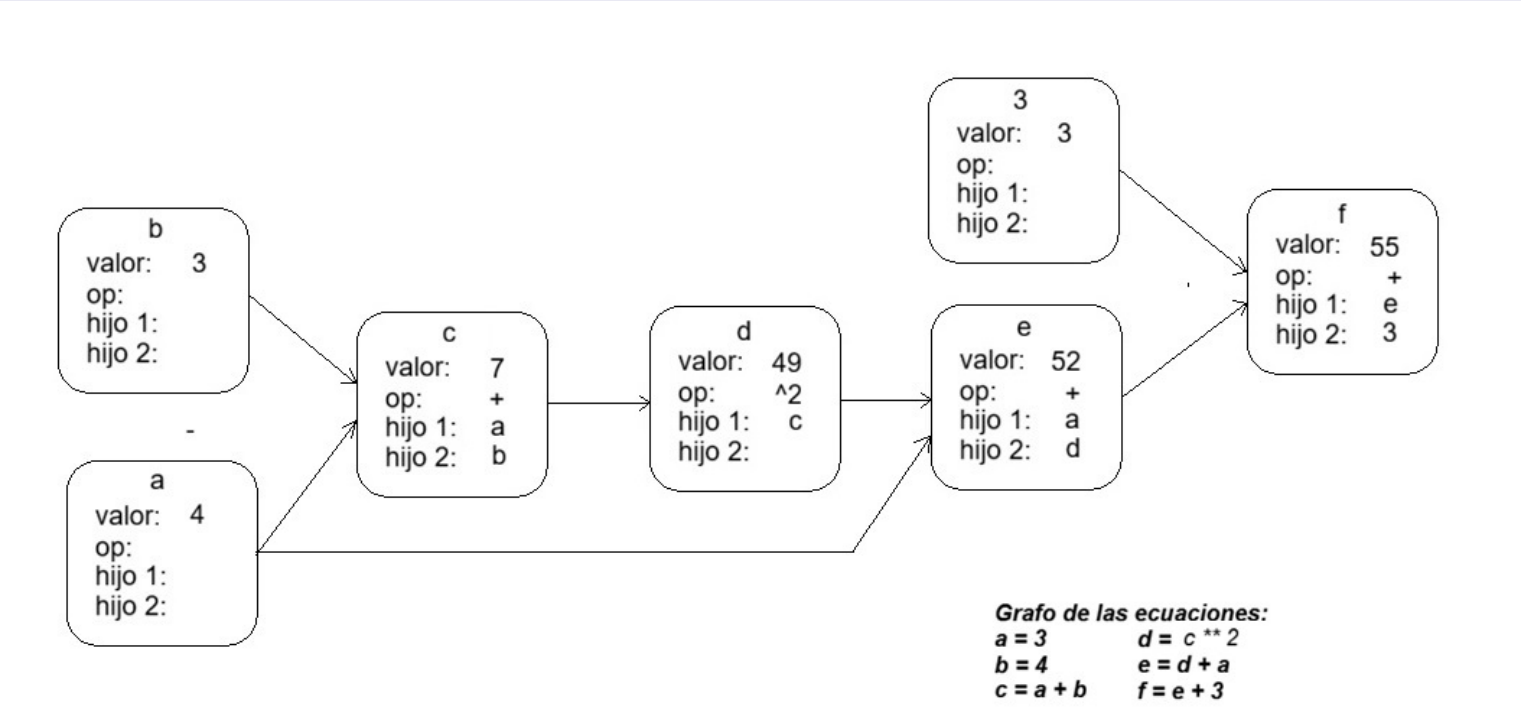
Calculo de las componentes del gradiente

- Consideremos un ejemplo un poco más completo:

$$a = 3$$
$$b = 4$$
$$c = a + b$$
$$d = c^{**}2$$
$$e = d + a$$
$$f = e + 3$$

Autograd

Calculo de las componentes del gradiente



Autograd

Calculo de las componentes del gradiente

- Supongamos que quisiéramos obtener la derivada parcial de f respecto a d.
- Teniendo en cuenta que

$$\begin{array}{ll} a = 3, & d = c^{**}2 \\ b = 4, & e = d + a \\ c = a + b, & f = e + 3 \end{array}$$

entonces
$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial e} * \frac{\partial e}{\partial d} = 1 * 1 = 1$
donde aplicamos la regla de la cadena.

- También podríamos haber escrito
$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial f} * \frac{\partial f}{\partial e} * \frac{\partial e}{\partial d} = 1 * 1 * 1 = 1$

Autograd

Calculo de las componentes del gradiente

- Nuevamente, teniendo en cuenta que

$$\begin{array}{ll} a = 3, & d = c * c \\ b = 4, & e = d + a \\ c = a + b, & f = e + 3 \end{array}$$

- ¿Qué pasa si queremos obtener la derivada $\frac{\partial f}{\partial a}$?

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial f} * \frac{\partial f}{\partial e} * (\frac{\partial e}{\partial d} * \frac{\partial d}{\partial c} * \frac{\partial c}{\partial a} + \frac{\partial e}{\partial a}) =$$
$$1 * 1 * (1 * 2c * 1 + 1) = 2c + 1$$

- Como $c = a + b$, $c = 7 \implies 2c + 1 = 15$.

Autograd

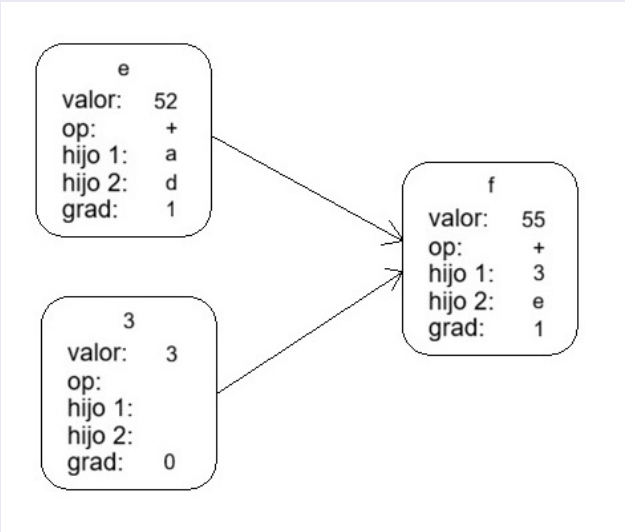
Calculo de las componentes del gradiente

- La idea de Autograd es hacer esto en forma automática valiéndose del grafo construido y algún detalle más.
- Lo primero que vamos a hacer es agregar a cada nodo un elemento más que lo denominaremos grad y que será la componente del gradiente correspondiente a la variable del nodo.

Autograd

Calculo de las componentes del gradiente

- Así, para los nodos e, 3 y f tendremos:

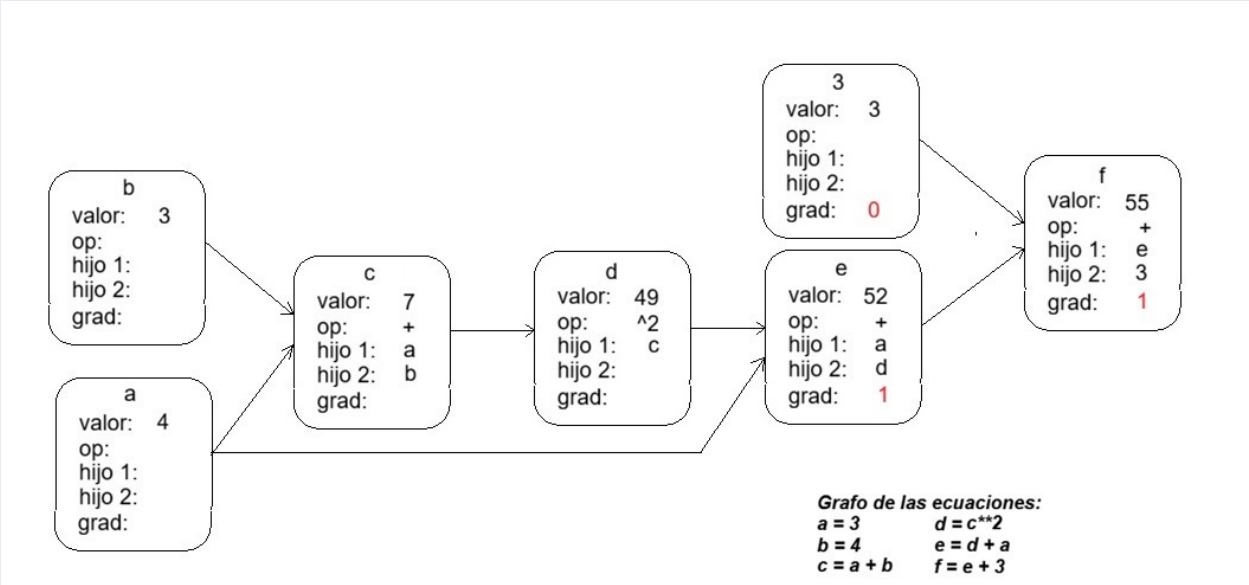


- Dado que $\frac{\partial f}{\partial f} = 1$, grad de f vale 1.
- Si derivamos f respecto a e (pensar en $f = e + 3$) la derivada es $\frac{\partial f}{\partial e} = \frac{\partial f}{\partial f} * \frac{\partial f}{\partial e} = 1$

Autograd

Calculo de las componentes del gradiente

- Veamos el grafo completo con los elementos **grad** en los nodos que calculamos:



Autograd

Calculo de las componentes del gradiente

- La idea es la siguiente: parto del nodo raíz, que en este caso es f, le asigno a grad el valor 1 (f.grad $= 1$).
- Luego de acuerdo a la operación que involucra a dicho nodo (en este caso es $+$), calculo la derivada de f respecto a cada uno de los operandos (hijos de f, en este caso e y 3).
- Así completo el valor de grad en los nodos de los hijos: e.grad $=$ f.grad * $\frac{\partial f}{\partial e}$, y 3.grad $=$ f.grad * $0 = 0$

Autograd

Calculo de las componentes del gradiente

- Luego sigo con cada uno de los hijos: e y 3.
- Cuando llego a e me fijo la operación asociada, en este caso es $+$.
- Los hijos de e son a y d, así que calculo a.grad:
  $a.grad = e.grad * \frac{\partial e}{\partial a} + c.grad * \frac{\partial c}{\partial a}$ y
  $d.grad = e.grad * \frac{\partial e}{\partial d}$
  y asigno un valor al elemento grad del nodo a y del nodo d.
- Fijarse que para a.grad consideramos una suma ya que a aporta a e pero también a c (veremos más adelante cómo implementar esto).

Autograd

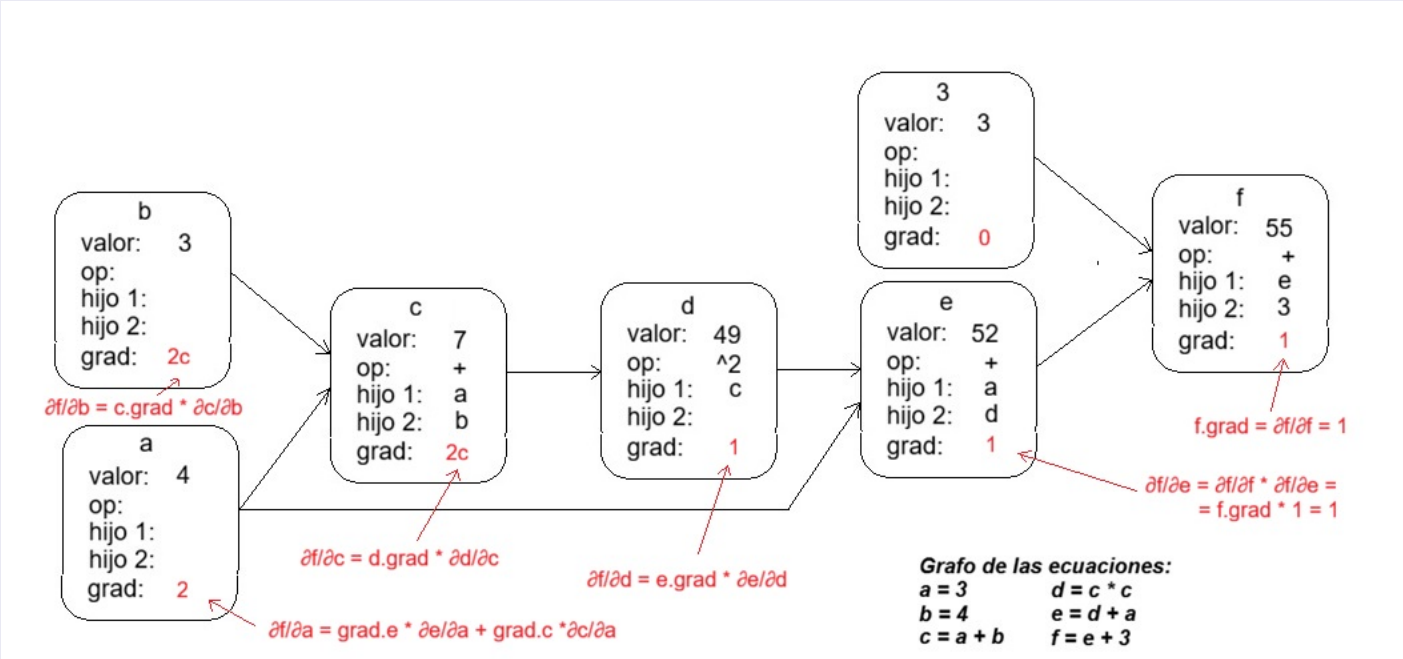
Calculo de las componentes del gradiente

- El nodo d tiene 1 solo hijo que es c.
- La operación en d es la potencia y para calcular el gradiente grad de c procedo según
$$c.grad = d.grad * \frac{\partial d}{\partial c}.$$
- De manera similar operamos sobre el nodo b.
- Fijarse que para actualizar el valor de grad en cada nodo partimos de f y procedemos hacia atrás.

Autograd

Calculo de las componentes del gradiente

- Finalmente obtenemos:



Autograd

Calculo de las componentes del gradiente

- Lo que vamos a hacer ahora es completar la definición de la clase Var definiendo una función (método) que pueda calcular el valor de grad de cada uno de los hijos.
- Como el cálculo de grad varía de acuerdo a la operación con su(s) hijo(s) haremos que dicho cálculo dependa de dicha operación.

Autograd

Calculo de las componentes del gradiente

- Para el caso que la operación sea la suma:

```
def _retroceso(self):
    if self._op == "+":
        list(self._hijos)[0].grad += self.grad * 1
        list(self._hijos)[1].grad += self.grad * 1
    informar(list(self._hijos)[0], self, list(self._hijos)[1])
    return
```

Autograd

Calculo de las componentes del gradiente

- Por comodidad introducimos informar():

```
def informar(hijo_0: Var, ppal: Var, hijo_1: Var = None):
    if ppal._op == '+' or ppal._op == '*' or ppal._op == 'tanh':
        map_ = {"+": "Suma", "*": "Multiplicación", "tanh": "Tanh"}
        if hijo_1:
            print(f"{ppal.nombre} como {map_[ppal._op]} entre
                {hijo_0.nombre} y {hijo_1.nombre}")
        else:
            print(f"{ppal.nombre} como {map_[ppal._op]}
                ({hijo_0.nombre})")
    else:
        print(f"Ir para atrás desde {ppal.nombre}")
        print("————————")
        print(f"{ppal.nombre} — Grad:", {ppal.grad}
        print(f"{hijo_0.nombre} — Grad: ", {hijo_0.grad}
        if hijo_1 is not None:
            print(f"{hijo_1.nombre} — Grad: ", hijo_1.grad)
        print(" ")
```

Autograd

Calculo de las componentes del gradiente

- ¿Cómo se vería todo esto en un programa Python?

```
# Grafo
a = Var(2, nombre = "a")
b = Var(3, nombre = "b")
c = a + b
c.nombre = "c"
d = a * c
d.nombre = "d"
# Retropropagar
d.grad = 1.0
d._retroceso()
a._retroceso()
c._retroceso()
```

Autograd

Calculo de las componentes del gradiente

- A los efectos de tener una manera práctica de visualizar el grafo y el valor de grad para cada una de las variables, usaremos una función para dibujar dicho grafo con el valor de las variables y de grad.
- Esta función la llamamos DibujarGrafo que podemos ver definida al final de estas diapositivas.

Autograd

Calculo de las componentes del gradiente

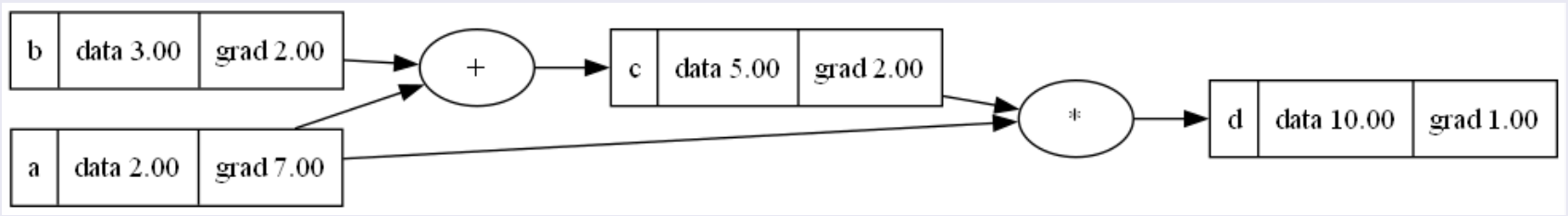
- Por ejemplo, al programa anterior le podemos agregar al final que dibuje el grafo con los valores de las variables y su valor de grad:

```
# Grafo
a = Var(2, nombre = "a")
b = Var(3, nombre = "b")
c = a + b
c.nombre = "c"
d = a * c
d.nombre = "d"
# Retropropagar
d.grad = 1.0
d._retroceso()
a._retroceso()
c._retroceso()
DibujarGrafo(d)
```

Autograd

Calculo de las componentes del gradiente

Y obtenemos el siguiente gráfico:



Autograd

Calculo de las componentes del gradiente

- Algo incómodo del programa que escribimos recién es que necesitamos llamar a la función _retroceso() para cada variable que interviene.
- Y peor aún, necesitamos saber el orden.

Autograd

Calculo de las componentes del gradiente

- Como el ejemplo es sencillo, no es difícil darse cuenta el orden. Pero en un grafo de miles de nodos sería poco práctico hacer el llamado de _retroceso() para cada variable en el orden adecuado.
- **Autograd** justamente fue pensado para resolver situaciones de cientos, miles o millones de cálculos.

Autograd

Calculo de las componentes del gradiente

- Ahora bien, ¿cómo establecer el orden en que deben ser invocadas las funciones _retroceso() para cada variable?
- Tener en cuenta que para calcular el valor de grad de los hijos de un nodo es necesario saber el valor de grad para dicho nodo.
- Lo que queda claro es que debemos empezar de atrás para adelante.
- En el ejemplo que recién vimos, debemos empezar por d.

Autograd

Calculo de las componentes del gradiente

- Una forma de recorrer todas las variable del grafo en el orden adecuado es siguiendo un **ordenamiento topológico**.
- Como el grafo es un digrafo y acíclico (GDA), podemos valernos de eso para saber qué nodos debimos haber calculado antes de calcular el valor de grad para un nodo dado.
- La forma de obtener un ordenamiento topológico está dado por el siguiente algoritmo (en la siguiente lámina escrito en Python):

Autograd

Calculo de las componentes del gradiente

```
def obtener_ordenamiento_topologico(root: Var):
    # Obtener el ordenamiento topológico para los nodos en el GDA
    topo = [ ]
    visitados = set()
    def construir_ordenamiento_topologico(v):
        # Agregar v si éste no está en el conjunto de visitados
        if v not in visitados:
            visitados.add(v)
            # Hacer recursión sobre cada uno de los hijos
            # hasta encontrar un nodo hoja
            # donde no se aplicará la recursión
            # y se devolverá una lista
            for hijo in v._hijos:
                construir_ordenamiento_topologico(hijo)
                topo.append(v)
    construir_ordenamiento_topologico(root)

    return topo
```

Autograd

Calculo de las componentes del gradiente

- Se puede observar que el nodo v con que se llama a la función construir_ordenamiento_topologico(v) la primera vez es el último en agregarse a la lista topo.

Autograd

Calculo de las componentes del gradiente

- Pero el único valor de gradiente que conocemos (y que es 1)
  es justamente el que corresponde al nodo con que llamamos la primera vez a la función construir_ordenamiento_topologico(v).

Autograd

Calculo de las componentes del gradiente

- Por lo tanto, debemos recorrer la lista topo en sentido inverso, esto es, recorriendo los nodos indicados por la lista topo pero de atrás para adelante.
- Recordemos la función _retroceso():

```
def _retroceso(self):
    if self._op == "+":
        list(self._hijos)[0].grad += self.grad * 1
        list(self._hijos)[1].grad += self.grad * 1
    informar(list(self._hijos)[0], self, list(self._hijos)[1])
    return
```

Autograd

Calculo de las componentes del gradiente

- ¿Cómo lo hacemos eso?

```
def retroceso(self):
    # Construir el ordenamiento topológico de acuerdo al GDA
    topo = [ ]
    visitados = set()
    def construir_ordenamiento_topologico(v):
        if v not in visitados:
        visitados.add(v)
        for hijo in v._hijos:
            construir_ordenamiento_topologico(hijo)
            topo.append(v)
    construir_ordenamiento_topologico(self)
    # Invertir la lista topo y proceder
    # hacia atrás un nodo a la vez aplicando la regla de la cadena
    # para obtener su gradiente
    self.grad = 1
    for v in reversed(topo):
        v._retroceso()
```

Autograd

Calculo de las componentes del gradiente

- Apliquemos el algoritmo completo para una red muy sencilla:
  - 2 entradas (x1 y x2)
  - 2 conexiones (w1 y w2) que excitan un único nodo de salida
  - la función de activación Tanh().
- Es decir, para un perceptrón simple no lineal.

(Ver programa 2 y programa 6 en el código python)

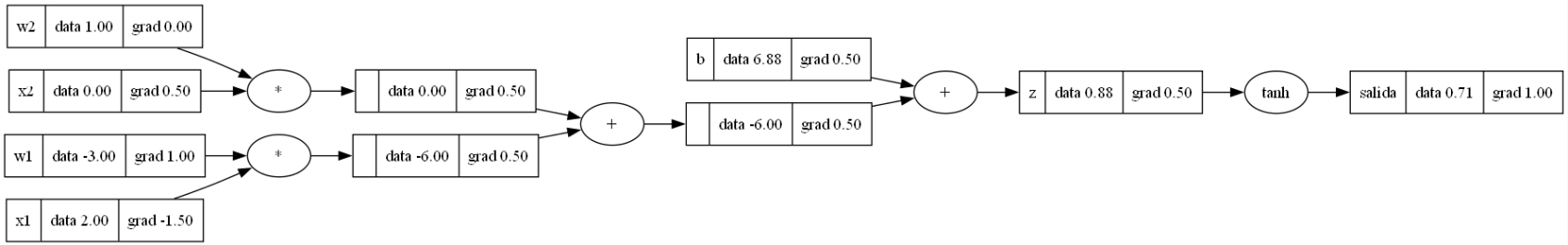
Autograd

Calculo de las componentes del gradiente

```
# Entradas x1 y x2
x1 = Var(2.0, "x1")
x2 = Var(0.0, "x2")
# Pesos
w1 = Var(-3.0, "w1")
w2 = Var(1.0, "w2")
b = Var(6.88, "b")
# Operaciones de multiplicación y suma
z = w1 * x1 + w2 * x2 + b
z.nombre = "z"
# Salida
salida = z.tanh()
salida.nombre = "salida"
retroceso(salida)
# Dibujar el grafo
DibujarGrafo(salida)
```

Autograd

Calculo de las componentes del gradiente



Autograd

Seudocódigo usando tensorflow

Seudocódigo

```
# Definir el modelo, cjto_entrenamiento, etc
...
epocas = N
tasa_apendizaje = 0.001
for epoca in range(epocas):
    for paso(x,y) in cjto_entrenamiento:
        with tf.GradientTape() as tape:
            logits = model(x)
            perdida = mae(logits, y)
        gradientes = tape.gradient(perdida, model.trainable_weigths)
    sgd.apply_gradients(gradientes, tasa_aprendizaje, model.trainable_weigths)
```

Autograd

Ejemplo muy simple

Ejemplito

```
import tensorflow as tf
x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y = x**2
print(tape.gradient(y, x).numpy())
```

Resultado:

6.0

Autograd

Código para la función DibujarGrafo()

```
def DibujarGrafo(root: Var):
    dot = Digraph(format = "png", graph_attr = {'rankdir': 'LR'})
    nodos, arcos = obtener_nodos_y_arcos(root)
    for nodo in nodos:
        # Definir la variable nodo_uuid que usa graphviz
        nodo_uuid = str(id(nodo))
        # Dibujar el nodo
        dot.node(name = nodo_uuid, label = "{ %s | data %.2f | grad %.2f}" %
                 (nodo.nombre, nodo.valor, nodo.grad), shape = "record")
        # Dibujar un nodo adicional donde está explicitada la operación
        if nodo._op:
            dot.node(name = nodo_uuid + nodo._op, label = f"{nodo._op }")
            dot.edge(tail_name = nodo_uuid + nodo._op, head_name = nodo_uuid)
    # Iterar sobre todos los arcos usando los nodos que vinculan cada arco
    for nodo_1, nodo_2 in arcos:
        dot.edge(tail_name = str(id(nodo_1)), head_name = str(id(nodo_2)) +
                 nodo_2._op)
        dot.render('Graph', view = True)
    return dot
```

Autograd

Código para la función obtener_nodos_y_arcos()

```
def obtener_nodos_y_arcos(root: Var):
    # Inicializar los conjuntos de arcos y nodos
    nodos, arcos = set(), set()
    # Definir la función recursiva
    def insertar(nodo_actual: Var):
        # Si el nodo no fue visitado, agregarlo
        if nodo_actual not in nodos:
            nodos.add(nodo_actual)
            # Iterar sobre los hijos
            for hijo in nodo_actual._hijos:
                arcos.add((hijo, nodo_actual))
                # Hacer recursión sobre cada hijo
                insertar(hijo)
    # Llamar a la función insertar sobre el nodo raíz
    insertar(root)
    return nodos, arcos
```